

FlashPort

How It Works

AI-Powered Customs Declaration Automation

OCR Pipeline | Risk Scoring | XGBoost Training

AI Open Innovation Challenge 2026 - Case 1: Cikarang Dry Port

Team: Teknik Logistik - President University

[MOBILE] Mobile App Offline-first camera scan	[BACKEND] FastAPI Backend OCR + AI extraction	[WEB] Web Dashboard Manager review + CEISA
--	--	---

This document explains the complete technical flow from the moment a field operator takes a photo of a trade document, to the AI extracting customs fields, scoring risk, and sending the result to the manager's dashboard. It also explains how the XGBoost risk model was trained.

PART 1 The OCR Pipeline

Every time an operator uploads a document (photo or PDF), the backend runs this exact sequence of steps. Nothing is manual the entire process from raw image to risk score happens automatically in under 3 seconds.

MOBILE		BACKEND		DATABASE
Take photo or attach file	POST /sync (base64 image)	OpenCV preprocess + Tesseract OCR	extract fields + score risk	save result + push to dashboard

1

Mobile sends the image

The operator's photo is compressed from ~3MB to ~300KB using `flutter_image_compress`, then converted to base64 text and sent to the backend as a JSON payload.

```
POST /sync
{
  "scan_id": "abc-123-uuid",
  "document_type": "commercial_invoice",
  "operator_id": "CDP-001",
  "image_b64": "/9j/4AAQSkZJRgAB..." your photo, base64 encoded
}
```

2

Detect file type PDF or image?

Base64 encoded files always start with a predictable header. The backend checks this to decide how to process it. PDFs need to be converted to images first.

```
# PDF base64 always starts with "JVBERi0" (that is %PDF- in base64)
# JPEG starts with "/9j/" PNG starts with "iVBORw"

if is_pdf(payload.image_b64):
    images = decode_pdf_pages(payload.image_b64) # one image per page
    text = " ".join(run_tesseract(preprocess(img)) for img in images)
else:
    text = run_tesseract(preprocess(decode_image(payload.image_b64)))
```

3

OpenCV preprocessing clean up the image

Raw photos are often blurry, skewed, or have bad lighting. OpenCV fixes this before Tesseract reads it, which dramatically improves accuracy.

Before preprocessing	After preprocessing
Color photo, shadows, slight angle, noise	Black & white, straight, clean ready for OCR
HS Code: 8471.30.00 might be read as H5 C0de: 8471.30.OO (0 vs O, l vs 1)	HS Code: 8471.30.00 clean text Tesseract reads this correctly every time

```
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY) # remove color
thresh = cv2.adaptiveThreshold(gray, 255, # make text pure black
cv2.ADAPTIVE_THRESH_GAUSSIAN_C, # background pure white
cv2.THRESH_BINARY, 11, 2)
# deskew: rotate image back to straight if taken at an angle
```

4

Tesseract OCR image to raw text

Tesseract reads the cleaned image and outputs everything it sees as plain text. This is not structured yet just a big string of characters, including all labels, values, headers, and even noise.

```
text = pytesseract.image_to_string(
    image,
    lang="eng+ind", # English + Indonesian language packs
    config="--psm 6" # treat as a uniform block of text
)
```

Example Tesseract output (raw):

```
COMMERCIAL INVOICE / Faktur Komersial
Invoice No.: INV-2026-5969 Date: 13 April 2026
SHIPPER / EXPORTER CONSIGNEE / IMPORTER
Apple Inc. PT LG Electronics Indonesia
One Apple Park Way, Cupertino Jl. Jend. Gatot Subroto Kav. 9-11
PORT OF LOADING PORT OF DISCHARGE CONTAINER NO.
Rotterdam Cikarang Dry Port YMLU7593824
HS Code Description Qty Unit Price Amount
8471.30.00 Portable Auto Data... 91 1,470.34 USD 133,800.94
Net Weight: 460.59 KG Gross Weight: 497.44 KG
```

NOTE: This is just raw text no structure, no fields. The regex extractor in Step 5 turns this into individual named values.

5

Regex field extractor find specific values

Regular expressions (regex) are patterns that scan the raw text and extract specific values. Each field has its own pattern tuned for customs document formats.

HS Code	<code>\d{4}[\.]\d{2}[\.]\d{2}</code>	4-2-2 digit structure	8471.30.00
Invoice Value	<code>USD\s*[\d,]+\.\d{2}</code>	Currency code + numbers	USD 133,800.94
Container ID	<code>[A-Z]{4}\d{7}</code>	4 letters + 7 digits	YMLU7593824
Importer	<code>CONSIGNEE[\s:]+([A-Z].*)</code>	Label then company name	PT LG Electronics Indonesia
Net Weight	<code>[\d.]+\s*KG</code>	Number + KG	460.59 KG
Vessel	<code>(?:MV MT SS)\s+\s+</code>	MV/MT/SS prefix	MV Yang Ming Wellness

After all patterns run, the result is a clean Python object with named fields:

```
ExtractionResult(
    hs_code = "8471.30.00",
    invoice_value = "USD 133,800.94",
    container_id = "YMLU7593824",
    importer = "PT LG Electronics Indonesia",
    exporter = "Apple Inc.",
    net_weight = "460.59 KG",
    vessel_name = "MV Yang Ming Wellness",
    port_of_origin = "Rotterdam, Netherlands",
    missing_critical = [] # nothing missing great!
)
```

6

XGBoost risk scoring predict the lane

The extracted fields are converted to 16 numbers (features) and fed into the XGBoost model. The model outputs probabilities for each lane and we compute a 0100 risk score.

Feature	Value (this doc)	Why it matters
has_hs_code	1 (present)	Missing HS code = immediate risk
has_invoice_value	1 (present)	Customs needs declared value
has_container_id	1 (present)	Can't track goods without it
has_importer	1 (present)	Who receives the goods
has_exporter	1 (present)	Who sent the goods
has_vessel	1 (present)	Which ship carried it
has_port	1 (present)	Country of origin
missing_field_count	0 (none)	Total critical fields missing
confidence_score	2 (high)	OCR quality: high=2 medium=1 low=0
is_restricted_hs	0 (no)	Weapons / chemicals = restricted
invoice_value_log	11.8	log(133,800) log scale handles large values
is_high_value	1 (>USD 50k)	High value shipments get extra scrutiny
high_val_no_cont	0 (has cont.)	High value without container = very suspicious
hs_high_scrutiny	1 (yes)	Electronics = moderate scrutiny category

```
# Model outputs probability for each lane:
proba = model.predict_proba([features])[0]
# [0.71, 0.22, 0.07]
#
# Green Yellow Red
# Convert to 0-100 risk score:
risk_score = P(yellow) × 40 + P(red) × 100
= 0.22 × 40 + 0.07 × 100
= 8.8 + 7.0 = 15.8 16% GREEN LANE
```

GREEN LANE	YELLOW LANE	RED LANE
Risk score < 30% All fields present Clean declaration Immediate release	Risk score 30-70% Some fields missing or medium confidence Document verification	Risk score > 70% Restricted HS code Watchlist hit / missing critical fields Physical inspection

7

Save to database + push to dashboard

Everything is saved to PostgreSQL including the original document image in base64 so the manager can view it. The result is broadcast live to the web dashboard via WebSocket, and a push notification is sent to the operator's phone.

```
# Save to PostgreSQL
declaration = Declaration(
    scan_id = "abc-123",
    hs_code = "8471.30.00",
    risk_score = 16,
```

```
risk_badge = "green",
image_data = "",
ceisa_ready = True,
)

# Push live to web dashboard
await broadcast({ "event": "new_declaration", "data": response })

# Return result to mobile app
return { "risk_score": 16, "risk_badge": "green", "ceisa_ready": True }
```

PART 2 How XGBoost Was Trained

The XGBoost risk model was trained on 600 synthetic customs declarations designed to reflect real patterns at Cikarang Dry Port. Here is exactly how the training data was built and how the model learned from it.

1

Generate 600 realistic fake declarations

A Python script generated records that look like real Cikarang Dry Port data with realistic company names, HS codes, container IDs, vessels, and ports.

Profile	Count	What it looks like	Label
Clean (55%)	330	All fields present, valid HS code, high confidence	Green
Partial (25%)	150	12 non-critical fields missing (vessel, port, carton count)	Yellow
Missing (12%)	72	24 critical fields missing (HS code, importer, value, container)	Red
Risky (8%)	48	Restricted HS code (weapons/chemicals), very high value, low confidence	Red

```
# Example clean record (Green):
{ "hs_code": "8471.30.00", "invoice_value": "USD 37,414.23",
  "container_id": "APHU6497677", "importer": "PT Garudafood Putra Putri Jaya",
  "exporter": "Posco Co., Ltd", "confidence": "high",
  "jalur": "green", "jalur_label": 0 }

# Example risky record (Red):
{ "hs_code": "9301.00.00", "invoice_value": "USD 2,069,981", "exporter": "",
  "confidence": "medium",
  "jalur": "red", "jalur_label": 2 }
```

2

Convert records to feature vectors

Every record becomes 16 numbers this is called a feature vector. XGBoost only understands numbers, not text.

```
# Clean record feature vector:
features = [
  has_hs_code=1, has_invoice=1, has_container=1, has_importer=1,
  has_exporter=1, has_vessel=1, has_port=1,
  missing_count=0, confidence=2, is_restricted=0,
  invoice_log=10.5, is_high_value=0, ... label: 0 (green)
]

# Risky record feature vector:
features = [
  has_hs_code=1, has_invoice=1, has_container=1, has_importer=1,
  has_exporter=0, has_vessel=1, has_port=1,
  missing_count=1, confidence=1, is_restricted=1,
  invoice_log=14.5, is_high_value=1, ... label: 2 (red)
]

# 600 rows × 16 columns = the training matrix
```

3

XGBoost builds decision trees

XGBoost (eXtreme Gradient Boosting) builds 300 decision trees one after another. Each tree learns to fix the mistakes of the previous one. This process is called boosting each tree "boosts" the accuracy of the model.

Simplified example of what one tree learned:

```

Tree #1:
Is hs_code restricted? (weapons/chemicals)
YES predict RED (high risk)
NO
Are more than 2 critical fields missing?
YES predict YELLOW or RED
NO
Is OCR confidence low?
YES predict YELLOW
NO predict GREEN

Tree #2 corrects mistakes Tree #1 made
Tree #3 corrects mistakes Tree #2 made
... (300 trees total)

```

4

Results 89.5% accuracy on cross-validation

We tested the model using 5-fold cross-validation: split the 600 records into 5 groups, train on 4, test on 1, repeat 5 times. Average accuracy was 89.5%.

Metric	Value	Meaning
CV Accuracy	89.5%	Correct prediction 9 out of 10 times on unseen data
Green precision	100%	When model says Green always Green
Yellow recall	100%	Catches every Yellow declaration
Red recall	97%	Misses 2 Red out of 78 (predicted Yellow instead)
Training time	< 3 sec	Fast to retrain when you get real data

Top feature importances what the model cares about most:

Feature	Importance	Bar	Why
is_restricted_hs	36.5%		Restricted HS code (weapons, chemicals) is by far the strongest signal
hs_high_scrutiny	10.7%		Electronics, petroleum categories moderate scrutiny
missing_field_count	8.0%		How many critical fields are empty
has_port	6.0%		Port of origin present or not
has_container_id	4.6%		Container ID present or not
high_val_no_container	4.6%		High value shipment without container ID
has_hs_code	4.5%		HS code present or not
has_vessel	4.0%		Vessel name present or not

5

Model saved loaded once at startup

The trained model is saved to a file and loaded into memory when the backend starts. Every scan after that uses the model instantly no re-training needed per scan.

```

# Save after training:
model.save_model("backend/models/risk_model.xgb")

```

```
# Load once when backend starts (risk_scorer.py):
_MODEL = xgb.XGBClassifier()
_MODEL.load_model("backend/models/risk_model.xgb")

# Use on every scan (instant):
proba = _MODEL.predict_proba([features])[0] # [p_green, p_yellow, p_red]
```


PART 3 How to Retrain the Models

Both the XGBoost risk model and the spaCy NER model can be retrained at any time by running four simple scripts. The whole process takes under 2 minutes. Run them in this order:

1

Regenerate synthetic training data (600 declarations)

Generates a fresh set of 600 realistic Indonesian customs declarations covering Commercial Invoice, Bill of Lading, and Packing List. Uses real company names, HS codes, container IDs, vessels, and ports.

```
python scripts/generate_training_data.py
# Output: data/training_declarations.csv
# 600 records: 395 Green / 127 Yellow / 78 Red
# Distribution: Clean 55% | Partial 25% | Missing 12% | Risky 8%
```

2

Retrain XGBoost risk model

Reads the CSV, builds 16 feature vectors per record, runs 5-fold cross-validation, trains the final model on all 600 records, and saves to disk.

```
python scripts/train_risk_model.py
# Output: models/risk_model.xgb
# models/model_info.json
# CV accuracy: 89.5% | Training time: < 3 seconds
```

3

Generate sample documents — 100 PDFs + 100 PNGs per type

Creates realistic Indonesian customs documents as actual PDF and PNG files. These are used to test and train the OCR extraction pipeline on real document layouts. Total: 300 PDFs + 300 PNGs across all three document types.

```
python scripts/generate_sample_docs.py
# Output: data/sample_docs/
# commercial_invoice/
# docs/ <- 100 PDFs (real text layer, pypdf extracts directly)
# images/ <- 100 PNGs (300 DPI, Tesseract OCR ready)
# bill_of_lading/
# docs/ <- 100 PDFs
# images/ <- 100 PNGs
# packing_list/
# docs/ <- 100 PDFs
# images/ <- 100 PNGs
# Uses: real PT company names, HS codes, vessels, ports, container IDs
```

4

Regenerate NER training examples (300 annotated texts)

Generates 300 annotated text examples from the same data pools — covering English and Indonesian label variations for all 11 entity types.

```
python scripts/generate_ner_training.py
# Output: data/ner_training.spacy
# 300 examples across: commercial invoice, bill of lading, packing list
# Entity distribution: CONTAINER_ID 296 | HS_CODE 298 | IMPORTER 296 | ...
```

5

Retrain spaCy NER model

Fine-tunes `en_core_web_sm` on the 300 annotated examples. Uses early stopping so it stops automatically when it reaches peak accuracy.

```
python scripts/train_ner_model.py  
# Output: models/ner_model/  
# Typically converges at iteration 15  
# F1: 1.000 on all 11 entity types
```

6

Restart the backend — models load automatically

The backend loads both models once at startup. After retraining just restart the server — no code changes needed.

```
uvicorn app.main:app --host 0.0.0.0 --port 8000 --reload  
# Models are loaded once at startup:  
# XGBoost -> models/risk_model.xgb  
# spaCy NER -> models/ner_model/  
# Every scan after that uses both models instantly
```

NOTE: You can retrain as many times as you want. Each run generates fresh synthetic data with different random values from the same realistic data pools — so the model sees variety each time. The training pipeline is fully automated end-to-end.

PART 4 Mobile Upload Flow + Document Types

When an operator takes a photo or attaches a file, this is exactly what happens on the mobile app and how the backend decides which processing path to use.

1

Operator uploads a document

The operator taps Take Photo or Attach File. The app shows a preview screen with a green Attached badge, the file name and size, and an Upload and Sync button.

State	What happens	What operator sees
Online	Image compressed, sent to backend, OCR runs, result returned	Upload progress animation, then Result screen with risk score
Offline	Image saved to local SQLite as Pending status	Snackbar: Saved as pending, auto-syncs when connected
After sync	Backend processes, result pushed to web dashboard via WebSockets	Dashboard updates: Scanned -> Synced -> Reviewed
Approved	Manager clicks Approve in dashboard, FCM push sent	Push notification: Declaration Approved
Rejected	Manager clicks Reject with note, FCM push sent	Push notification: Declaration Rejected + manager note

2

Backend decides: PDF or Image?

The very first thing the backend checks is whether the upload is a digital PDF or a photo/image. This determines which processing path to use.

```
# PDF base64 always starts with "JVBERi0" (= %PDF- in base64)
# JPEG starts with "/9j/" PNG starts with "iVBORw"

if is_pdf(payload.image_b64):
    # Path A: Digital PDF extract text directly (no OCR)
    text = extract_pdf_text_direct(payload.image_b64) # uses pypdf
    if not text:
        # Scanned/image PDF fall back to Tesseract
        images = decode_pdf_pages(payload.image_b64)
        text = run_tesseract(preprocess(img)) for img in images
else:
    # Path B: Photo/image smart preprocess then Tesseract
    text = run_tesseract(preprocess(decode_image(payload.image_b64)))
```

3

Smart image preprocessing

Not all images need the same treatment. A clean digital image needs only grayscale conversion. A dark or blurry phone photo needs the full pipeline.

Image type	White pixel ratio	Processing applied
Clean digital image (PDF render)	> 55% white pixels	Grayscale only -- Tesseract reads it perfectly as-is
Good phone photo (well-lit, flat)	> 55% white	Grayscale only -- good lighting means clean contrast
Dark/noisy phone photo	< 55% white	Grayscale + Gaussian denoise + adaptive threshold + deskew

4

Three supported document types

FlashPort supports all three standard customs trade documents. Each has different fields -- the extractor knows which fields to expect per type.

Document	Key fields extracted	Invoice value?
Commercial Invoice	HS Code, Invoice Value, Container ID, Importer, Exporter, Net/Gross Weight, Vessel, Port	Yes - USD/EUR amount
Bill of Lading	HS Code, Container ID, Importer, Exporter, Vessel, Port, Carton Count, Gross Weight	No - Bill of lading does not contain invoice value

Packing List	HS Code, Container ID, Importer, Exporter, Vessel, Port, Net/Gross Weight, Capacity	NO Captive Document does not contain invoice
--------------	---	--

NOTE: Missing invoice_value on Bill of Lading and Packing List is NOT an error -- those document types do not contain invoice values. The risk scorer accounts for this by document type so they are not penalised for this missing field.

Complete System Summary

Component	Technology	What it does
Mobile App	Flutter (Dart)	Offline-first capture, auto-sync, FCM notifications
PDF Text Extraction	pypdf	Reads digital PDF text layer directly -- no OCR needed
Image Preprocessing	OpenCV (smart pipeline)	Grayscale only for clean images, full pipeline for noisy photos
OCR Engine	Tesseract eng+ind	Converts preprocessed image to raw text
Field Extraction 1	spaCy NER (custom trained)	11 entity types, 100% F1, English + Indonesian
Field Extraction 2	Python Regex fallback	Fills gaps from NER, handles table-merge OCR patterns
Risk Scoring	XGBoost (300 trees)	89.5% CV accuracy, 16 features
Explainability	SHAP TreeExplainer	Per-feature contribution shown as bar chart in dashboard
Database	PostgreSQL 15	Stores declarations, images, audit trail, watchlist, rules
Web Dashboard	React 18 + Tailwind CSS	9-page pro dashboard: approve/reject, CEISA, SLA, audit
Push Notifications	Firebase FCM HTTP v1	Approve/reject decisions + CEISA lane results
CEISA Gateway	Mock FastAPI (Phase 1)	SP2-200 / SP2-412 / SP2-500 lane responses in English